

## 实验四 分类算法（1）

### （一）目的和要求

- （1）掌握分类算法的概念的使用场景
- （2）掌握使用机器学习分类方法的进行数据预测的有效方法
- （3）掌握模型效果的方法及不同模型性能的比较方法

### （二）实验知识准备

（1）高斯贝叶斯：高斯贝叶斯的基本思想是利用贝叶斯定理来计算给定样本属于各个类别的后验概率，并选择具有最高后验概率的类别作为该样本的预测类别。具体来说，它假设特征之间相互独立，并且每个特征的条件概率分布都服从高斯分布（正态分布）。

（2）决策树：决策树分类的基本原理是根据已有的数据样本，通过构建一个树状模型来预测新的数据样本的类别。这个树状模型通过一系列的特征测试（也称为节点），将数据样本逐级划分到不同的类别中。每个内部节点代表一个特征属性的测试，每个分支代表该测试的一个可能结果，而每个叶节点则代表一个最终的分类结果。

（3）模型评价：模型评价是评估模型性能、有效性和适用性的重要过程。这一过程涉及多个方面，包括模型的准确性、泛化能力、可解释性、计算效率等。

#### ■ 评价维度

##### 1. 准确性

A. 准确率（Accuracy）：模型预测结果与真实结果的匹配程度，即正确预测的样本数占总样本数的比例。

B. 精确率（Precision）：模型预测为正样本中真正为正样本的比例，衡量了模型预测为正样本的准确性。

C. 召回率（Recall）：真正的正样本中被模型正确预测为正样本的比例，衡量了模型对正样本的发现能力。

D. F1-Score：综合考虑了精确率和召回率的一个综合评价指标，用于衡量模型的综合性能。

## 2. 泛化能力

- A. 通过交叉验证、留出验证等方法评估模型在未知数据上的表现。
- B. ROC 曲线和 AUC 值也是评估模型分类性能的重要工具，AUC 值表示 ROC 曲线下的面积，用于度量模型分类的准确性。

## 3. 可解释性

- A. 模型的可解释性对于理解模型决策过程、调试模型以及满足法规要求都至关重要。
- B. 对于复杂的机器学习模型，如深度学习模型，可能需要通过特征重要性分析、可视化等方法来提高其可解释性。

## 4. 计算效率

- A. 模型的训练时间和预测时间也是评价模型性能的重要指标。
- B. 在实际应用中，需要选择能够在合理时间内完成训练和预测的模型。

## ■ 评价方法

### 1. 交叉验证法

将原始数据分为  $K$  个不相交的子集，然后进行  $K$  次训练和验证。这种方法可以减小偶然误差，使评估更全面。

### 2. 留出验证法

通过将原始数据集分为训练集和测试集，然后使用训练集训练模型，测试集评估模型。这种方法可以真实地反映模型在未知数据上的性能，且易于理解和实施。

### 3. 自助法

通过有放回地从原始数据中抽样构成训练集和测试集。这种方法可以最大化地利用数据资源，特别适合样本容量不大的情况。

### 4. 混淆矩阵

评估分类模型性能的重要工具，提供了真正例、假正例等详细信息，可用于多分类问题的评估。

## （三）实验内容及步骤

### （1）使用贝叶斯算法识别葡萄酒类别

葡萄酒识别数据集（Wine Recognition dataset）通常用于多类别分类问题建

模。数据集包括从三个不同的品种（类别）的葡萄酒中测得的 13 种不同的化学特征，共 178 个样本。这些化学特征包括酸度、灰分、酒精浓度等。该数据集是由 UCI 机器学习库提供，并且已经被广泛用于分类和聚类任务，作为基准测试数据集之一，数据集地址：[Wine – UCI Machine Learning Repository](https://archive.ics.uci.edu/ml/datasets/wine)。数据集的说明如表 1。

表 1 葡萄酒数据集说明

| 列名                           | 描述                                                   |
|------------------------------|------------------------------------------------------|
| Alcohol                      | 酒精含量百分比                                              |
| Malic acid                   | 苹果酸含量（克/升）                                           |
| Ash                          | 灰分含量（克/升）                                            |
| Alcalinity of ash            | 灰分碱度（以 mEq/L 为单位）                                    |
| Magnesium                    | 镁含量（毫克/升）                                            |
| Total phenols                | 总酚含量（以毫克/升为单位）                                       |
| Flavanoids                   | 类黄酮含量（以毫克/升为单位）                                      |
| Nonflavanoid phenols         | 非类黄酮酚含量（以毫克/升为单位）                                    |
| Proanthocyanins              | 原花青素含量（以毫克/升为单位）                                     |
| Color intensity              | 颜色强度（以 absorbance 为单位，对应于 1cm 路径长度处的相对宽度）            |
| Hue                          | 色调，即色彩的倾向性或相似性（在 1 至 10 之间的一个数字）                     |
| OD280/OD315 of diluted wines | 稀释葡萄酒样品的光密度比值，用于测量葡萄酒中各种化合物的浓度                       |
| Proline                      | 脯氨酸含量（以毫克/升为单位），是一种天然氨基酸，与葡萄酒的品质和口感有关                |
| class                        | 类别（目标变量）代表三种不同品种的葡萄酒，通常标记为 class_0、class_1 和 class_2 |

表 1 中数据集中的类别（目标变量）代表三种不同品种的葡萄酒，在实际应用中，这些类别可能会被赋予更具描述性的名称，如琴酒、雪莉和贝尔莫得等。

1. 步骤一，导入本项目所需要的模块和包，如图 1 所示。

```
1 # 导入所需模块
2 import matplotlib.pyplot as plt
3 from sklearn.naive_bayes import GaussianNB
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import accuracy_score, confusion_matrix,
  classification_report, ConfusionMatrixDisplay
6 from sklearn.metrics import roc_curve, auc
7 from sklearn.datasets import load_wine
```

图 1 导入所需模块

2. 步骤二,使用 `load_data()`函数从 Keras 库中导入葡萄酒数据集到本地计算机中。  
参考代码如图 2 所示。

```
1 # 导入数据, 分别为输入特征和标签
2 wine = load_wine()
3 X=wine.data
4 Y=wine.target
5 # 划分训练集和测试集
6 X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2,
  random_state=42)
7 # 输出结果
8 print("训练集特征形状:", X_train.shape)
9 print("测试集特征形状:", X_test.shape)
10 print("训练集标签形状:", y_train.shape)
11 print("测试集标签形状:", y_test.shape)
```

图 2 从 Keras 加载数据集

3. 步骤三, 可视化训练集数据样本分布情况, 主要代码如图 3 所示。

```

1 # 可视化训练集样本数据分布
2 feature_names = wine.feature_names
3 target_names = wine.target_names
4 # 绘制散点图
5 plt.figure(figsize=(10, 6))
6 # 使用前两个特征进行绘图 (如 sepal length 和 sepal width)
7 scatter = plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap='viridis',
                        edgecolor='k', s=100)
8 # 设置图形的标签和标题
9 plt.xlabel(feature_names[0]) # Sepal length
10 plt.ylabel(feature_names[1]) # Sepal width
11 plt.title('Training Samples Scatter Plot')
12 # 添加图例
13 # 生成图例: 每个类别一个点
14 handles = [plt.Line2D([0], [0], marker='o', color='w',
                        markerfacecolor=scatter.cmap(scatter.norm(i)), markersize=10) for i in
             range(len(target_names))]
15 plt.legend(handles, target_names, title="Classes")
16 plt.grid()
17 plt.show()

```

图 3 可视化训练集样本分布

训练集样本分情况如图 4 所示:

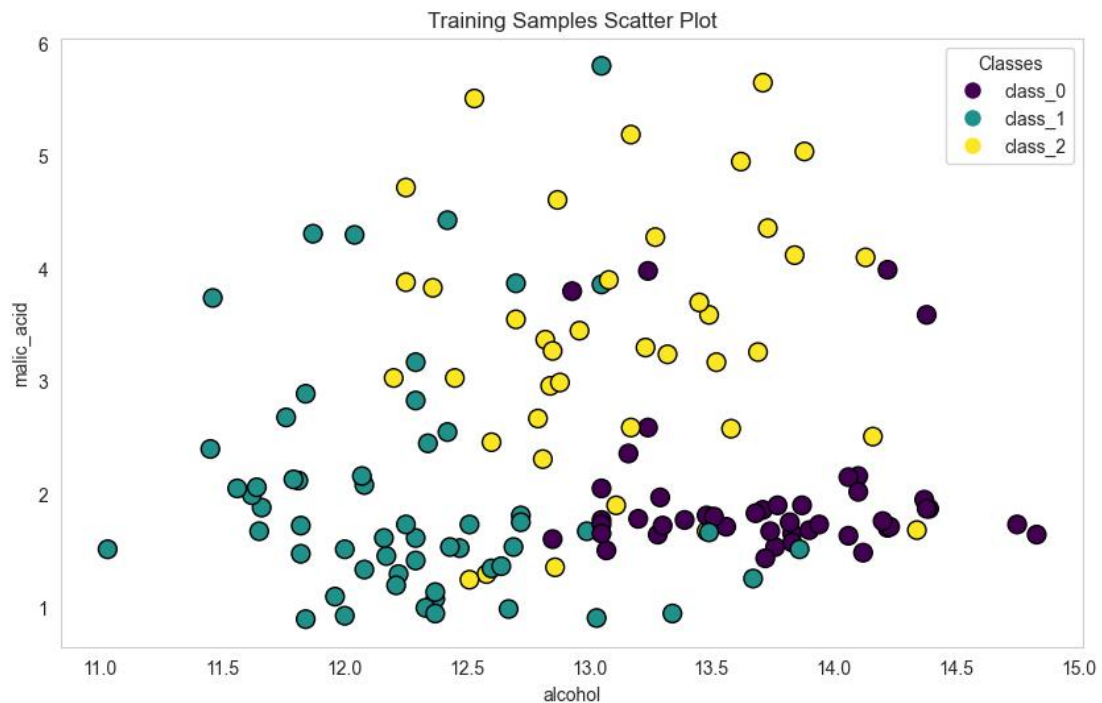


图 4 训练集样本分布情况

4. 步骤四, 建立并训练贝叶斯分类器, 进行预测并评估模型, 并输出模型的评

价指标，示例代码如图 5 所示。

```
1 #定义和训练贝叶斯模型
2 nb_model = GaussianNB()
3 nb_model.fit(X_train, y_train)
4 # 在测试集上进行预测
5 nb_y_pred = nb_model.predict(X_test)
6 # 获取预测概率
7 nb_y_pred_proba = nb_model.predict_proba(X_test)
8 # 评估模型
9 accuracy = accuracy_score(y_test, nb_y_pred)
10 conf_matrix = confusion_matrix(y_test, nb_y_pred)
11 class_report = classification_report(y_test, nb_y_pred, target_names=target_names)
12 # 打印评价指标
13 print(f"Accuracy: {accuracy:.2f}")
14 print("Confusion Matrix:\n", conf_matrix)
15 print("Classification Report:\n", class_report)
16 # 绘制混淆矩阵
17 ConfusionMatrixDisplay(confusion_matrix=conf_matrix,
    display_labels=target_names).plot(cmap='Blues')
18 plt.title('Confusion Matrix')
19 plt.show()
```

图 5 定义并训练分类器

分类器的混淆矩阵如图 6 所示。

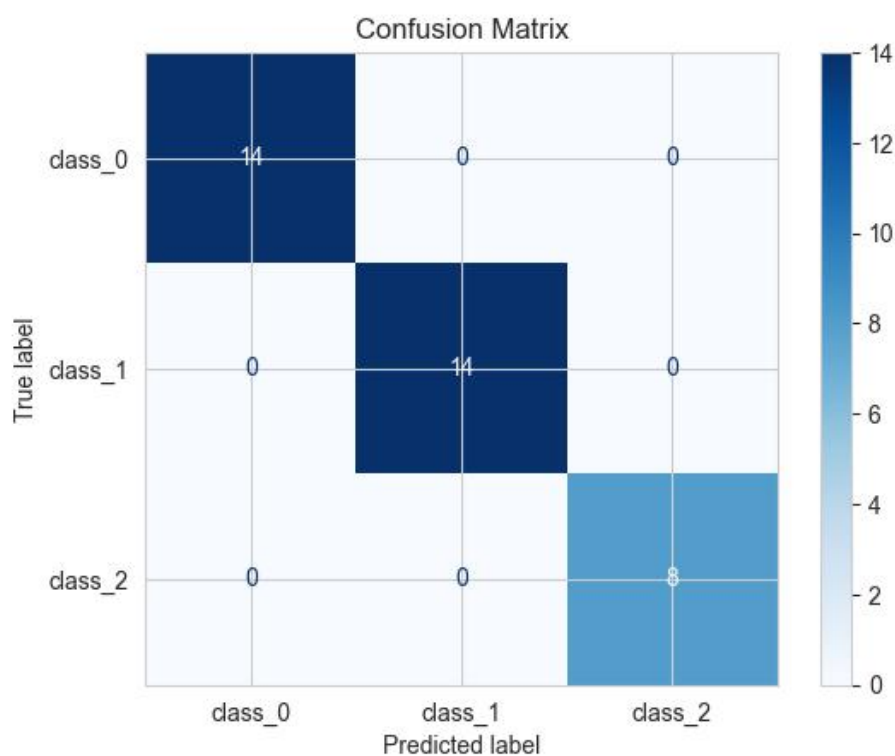


图 6 分类器混淆矩阵

5. 步骤五，绘制 ROC 曲线，评估分类器效果，示例代码如图 7。

```
1 # 绘制AUC-ROC曲线
2 plt.figure(figsize=(10, 6))
3 # 计算 ROC 曲线
4 fpr = dict()
5 tpr = dict()
6 roc_auc = dict()
7 n_classes = len(target_names)
8 for i in range(n_classes):
9     fpr[i], tpr[i], _ = roc_curve(y_test, nb_y_pred_proba[:, i], pos_label=i)
10    roc_auc[i] = auc(fpr[i], tpr[i])
11
12 # 绘制每个类别的 ROC 曲线
13 for i in range(n_classes):
14     plt.plot(fpr[i], tpr[i], label=f'ROC curve of class {target_names[i]} (area =
15         {roc_auc[i]:.2f})')
16 # 绘制随机猜测基线
17 plt.plot([0, 1], [0, 1], 'k--', label='Random')
18 # 设置图形的标签和标题
19 plt.xlabel('False Positive Rate')
20 plt.ylabel('True Positive Rate')
21 plt.title('Receiver Operating Characteristic (ROC) Curve')
22 plt.legend(loc='lower right')
23 plt.grid()
24 plt.show()
```

图 7 ROC 曲线

ROC 曲线，如图 8 所示。

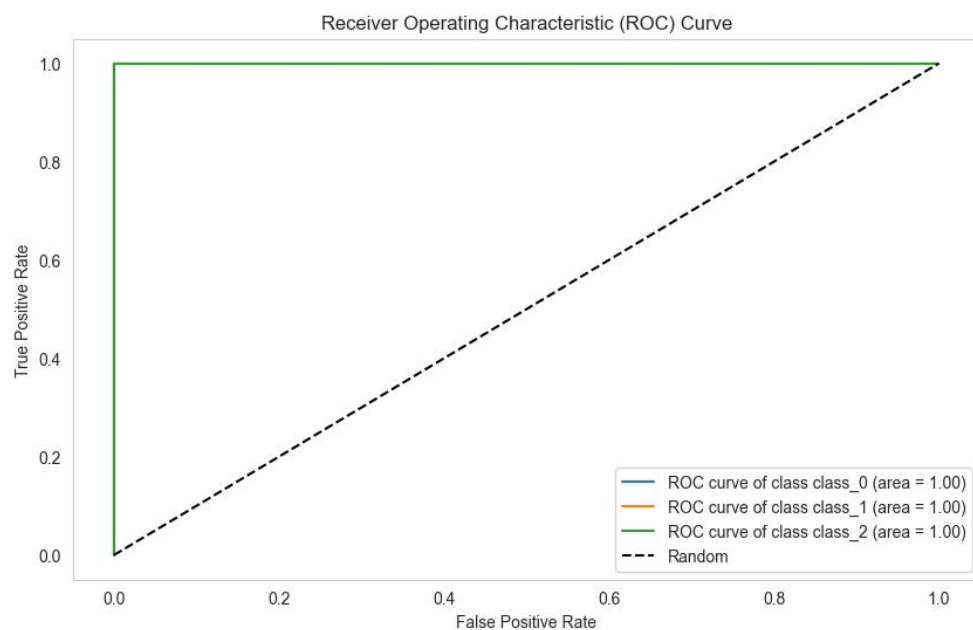


图 8 ROC 曲线



从图 8 可以看出 AUC 值为 1，说明模型性能良好。

6. 步骤六，定义并训练决策树模型以用于对比，示例代码如图 9 所示。

```
1 from sklearn.tree import DecisionTreeClassifier
2 # 训练决策树模型
3 dt_model = DecisionTreeClassifier(random_state=42)
4 dt_model.fit(X_train, y_train)
5 # 使用决策树进行预测
6 dt_pred = dt_model.predict(X_test)
```

图 9 定义并训练决策树模型

7. 步骤七，输出模型分类结果的评价指标，示例代码如图 10 所示。

```
1 # 计算准确率
2 dt_accuracy = accuracy_score(y_test, dt_pred)
3 # 评估模型
4 dt_conf_matrix = confusion_matrix(y_test, dt_pred)
5 print(f"Decision Tree Accuracy: {dt_accuracy:.2f}")
6 # 打印分类报告
7 print("Decision Tree Classification Report:\n", classification_report(y_test,
    dt_pred, target_names=target_names))
```

图 10 输出评价指标

8. 步骤八，绘制决策树的混淆矩阵，示例代码如图 11 所示。

```
1 # 绘制混淆矩阵
2 plt.figure(figsize=(12, 6))
3 ConfusionMatrixDisplay(confusion_matrix=dt_conf_matrix,
    display_labels=target_names).plot(cmap='Blues', ax=plt.gca())
4 plt.title('Decision Tree Confusion Matrix')
5 plt.tight_layout()
6 plt.show()
```

图 11 绘制决策树的混淆矩阵

混淆矩阵如图 12 所示，



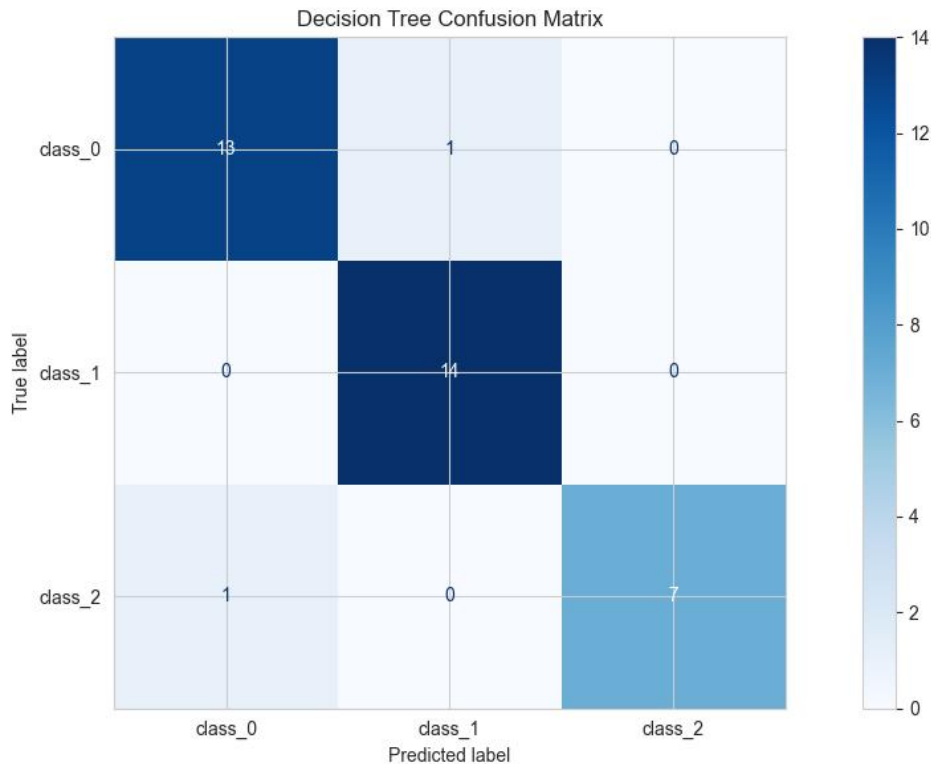


图 12 决策树的混淆矩阵

9. 步骤九，比对贝叶斯分类和决策树分类的 ROC 曲线，示例代码如图 13。

```

1 # 获取预测概率
2 dt_y_pred_proba = dt_model.predict_proba(X_test)
3 # 计算 ROC 曲线
4 fpr_nb, tpr_nb, _ = roc_curve(y_test, nb_y_pred_proba[:, 1], pos_label=1)
5 roc_auc_nb = auc(fpr_nb, tpr_nb)
6 fpr_dt, tpr_dt, _ = roc_curve(y_test, dt_y_pred_proba[:, 1], pos_label=1)
7 roc_auc_dt = auc(fpr_dt, tpr_dt)
8
9 # 绘制 AUC-ROC 曲线
10 plt.figure(figsize=(10, 6))
11 # 绘制朴素贝叶斯 ROC 曲线
12 plt.plot(fpr_nb, tpr_nb, label=f'Naive Bayes (area = {roc_auc_nb:.2f})')
13 # 绘制决策树 ROC 曲线
14 plt.plot(fpr_dt, tpr_dt, label=f'Decision Tree (area = {roc_auc_dt:.2f})')
15 # 绘制随机猜测基线
16 plt.plot([0, 1], [0, 1], 'k--', label='Random')
17 # 设置图形的标签和标题
18 plt.xlabel('False Positive Rate')
19 plt.ylabel('True Positive Rate')
20 plt.title('Receiver Operating Characteristic (ROC) Curve')
21 plt.legend(loc='lower right')
22 plt.grid()
23 plt.show()

```

图 13 比较分类器性能

ROC 曲线的比较结果如图 14 所示。

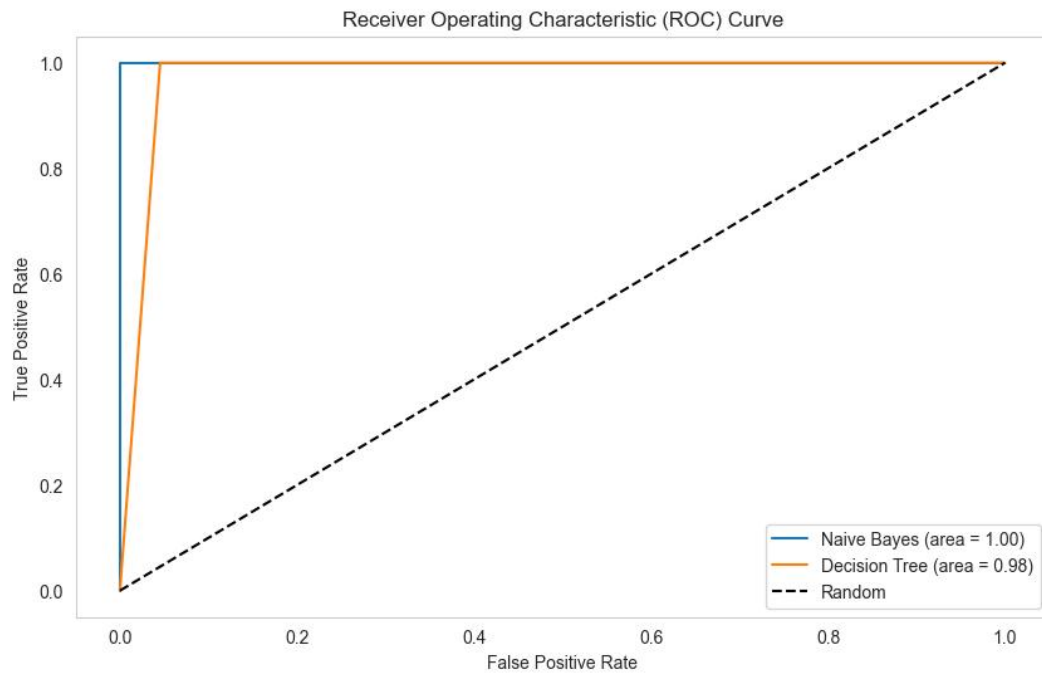


图 14 两个分类器 ROC 曲线

10. 步骤十，使用交叉验证评估不同模型的精度，示例代码如图 15。

```
1 import pandas as pd
2 from sklearn.model_selection import cross_val_score
3 # 进行交叉验证以比较模型的准确性
4 models = {
5     'Naive Bayes': nb_model,
6     'Decision Tree': dt_model
7 }
8 results = []
9 for model_name, model in models.items():
10     scores = cross_val_score(model, X_test, y_test, cv=5) # 5折交叉验证
11     results.append(scores)
12 # 将结果转换为 DataFrame 以便于绘图
13 results_df = pd.DataFrame(results, index=models.keys()).T
14
15 # 绘制箱线图
16 plt.figure(figsize=(10, 6))
17 sns.boxplot(data=results_df)
18 plt.title('Model Accuracy Comparison')
19 plt.xlabel('Model')
20 plt.ylabel('Accuracy')
21 plt.xticks(rotation=45)
22 plt.grid()
23 plt.show()
```

图 15 交叉验证代码

交叉验证后使用箱图可视化不同模型的精度对比，见图 16。

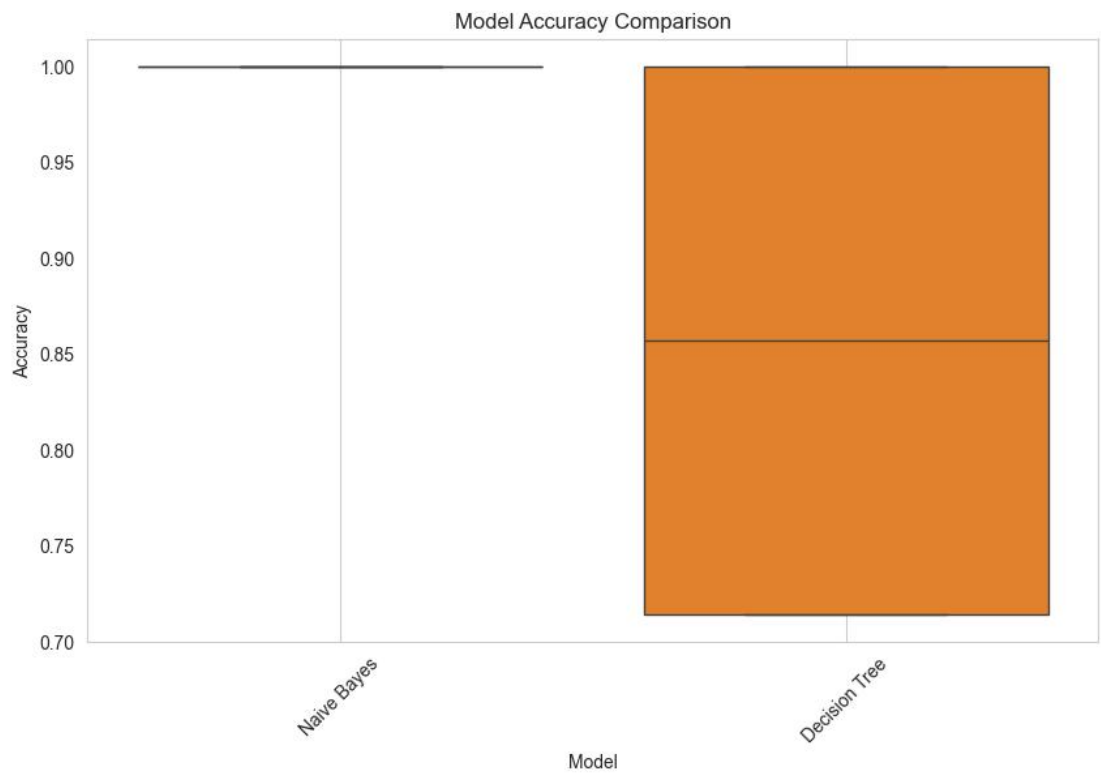


图 15 箱图比对分类精度

## (2) 实训项目

导入 `scikit-learn` 库中的鸢尾花数据集，使用三种不同的决策树方法（ID3，C4.5，CART）对鸢尾花数据进行分类。鸢尾花数据集各列说明如表 2 所示。

表 2 鸢尾花数据集说明

| 字段名称        | 字段类型 | 字段说明                              |
|-------------|------|-----------------------------------|
| SepalLength | 浮点型  | 萼片长度（厘米）                          |
| SepalWidth  | 浮点型  | 萼片宽度（厘米）                          |
| PetalLength | 浮点型  | 花瓣长度（厘米）                          |
| PetalWidth  | 浮点型  | 花瓣宽度（厘米）                          |
| Class       | 字符型  | 类别：Setosa, Versicolour, Virginica |

表 2 表明鸢尾花数据集中共有 150 条数据，分为 3 类，每类 50 条数据，每条数据包含 4 个特征值和 1 个标签值，其中特征值分别是花萼长度、花萼宽度、花瓣长度和花瓣宽度，标签值表示鸢尾花的种类，分别是 Setosa、Versicolor 和 Virginica，用整数 0、1 和 2 表示

1. 步骤一，导入需要的库文件，并使用 `load_iris()` 函数导入 `scikit-learn` 库中的鸢尾花数据集。
2. 步骤二，对数据进行初步探索，包括查看数据的基本信息和可视化特征之间的关系。
3. 步骤三，将数据集分为训练集和测试集，通常按 8:2 或 7:3 的比例划分。
4. 步骤四，使用训练集分别训练三个不同的分类器。
5. 步骤五，使用测试集进行预测，并将结果保存下来。
6. 步骤六，使用混淆矩阵和分类报告来评估并比较三个不同分类器的性能。
7. 步骤七，对比较结果进行可视化。